Predicting Music Popularity

Milestone 1: Task Exploration

TASK DESCRIPTION AND TYPE

I've chosen the **Song Popularity Dataset** for this project because it just grabbed my eye. The data is easily understandable, and it's easy to draw useful conclusions from it.

It's rated **10.00** in **Usability** on Kaggle, is **CC0: Public Domain**, and others described it as:

- Well-documented
- Well-maintained
- Clean data

There's one clear goal to achieve with this dataset: find out **what makes a song popular**. Starting from this premise we may find out what makes or breaks a songs success. This could mean drawing clear correlations between columns, like *danceability* and *song_popularity*, it could also mean finding unusual combinations of different components that affect a songs popularity.

We don't simply say a song is popular or not, we measure how popular is it. Since *song_popularity* is a numerical value (ranging from 0 to 100), this is a **regression task**. The goal will be to build a model that takes a song's features as input and outputs a predicted popularity score.

BACKGROUND OF THE DATASET

It's a **Kaggle Dataset**. Provided from **Google** and the collection methodology is listed as research. It has **99.7K views** and **15.7K downloads**. It was **last updated 4 years ago**, even though the expected update frequency is annual. This won't cause problems for the task at hand; there's one clear limitation that I think could affect this project: the budget for song promotion is not included. I have listened to many great songs that don't get the recognition they deserve and plenty of bad songs that nobody can stand on the radio. I'm sure there are many other factors that go into why a song becomes successful that do not appear in this dataset.

Why is the task relevant?

- Predicting a song's potential success can help in making decisions about **which tracks to promote** for artists and labels

- Understanding features that drive popularity can help **improve recommendation engines** and curate more **engaging playlists** for streaming services

- The data allows for analysis of **musical trends** and what qualities resonate the most with listeners

DESCRIBING THE DATASET

Let's load the first rows of data to get a first look:

song_name	song_popularity	song_duration_ms	...	time_signature	audio_valence
Boulevard of...	73	262333	...	4	0.474
In The End	66	216933	...	4	0.370
Seven Nation Army	76	231733	...	4	0.324

Features and Target

- **Target Variable**: *song_popularity*. This is an integer score from 0-100, with higher values indicating a greater success.
- **Features**: The remaining columns can be broken down further:
    - **Identifier**: *song_name* (I will likely drop this for modelling, not much correlation between names and popularity and even if there is, that's too many dimensions, it would just cause noise).
    - **Audio Features**: These are numerical metrics that describe the different qualities of a track.
        - ***song_duration_ms***: The duration of the song in milliseconds.
        - ***acousticness***: A value from 0.0 to 1.0 indicating how acoustic a song is.

        - ***danceability***: A value from 0.0 to 1.0 describing how suitable a track is for dancing.
        - ***energy***: A measure of intensity and activity from 0.0 to 1.0.
        - ***instrumentalness***: A value from 0.0 to 1.0. Values closer to 1.0 indicate the track is fully instrumental.
        - ***liveness***: A value from 0.0 to 1.0 detecting the presence of an audience.
        - ***loudness***: The overall loudness in decibels (dB).
        - ***speechiness***: The presence of spoken words in a track.
        - ***tempo***: The estimated tempo in beats per minute (BPM).
        - ***audio_valence***: A measure from 0.0 to 1.0 describing the musical positiveness (e.g., happiness, cheerfulness).
    - **Categorical/Ordinal Features**:
        - ***key***: The musical key of the track, represented as an integer (0=C, 1=C♯/D♭, etc.).
        - ***audio_mode***: Indicates if the track is in a major (1) or minor (0) scale.
        - **time_signature**: The number of beats in each measure.

Some other useful information about the dataset:

- Number of samples: 18835
- Number of columns: 15
- Number of **duplicate** song entries: 6777 (For example 'Till I Collapse appears 3 times, the rows are completely duplicated, we will keep first occurrences)
- There are no missing values in any of the columns (**df.isnull().sum()**)
- No dedicated training and test set, we will need to perform the split ourselves

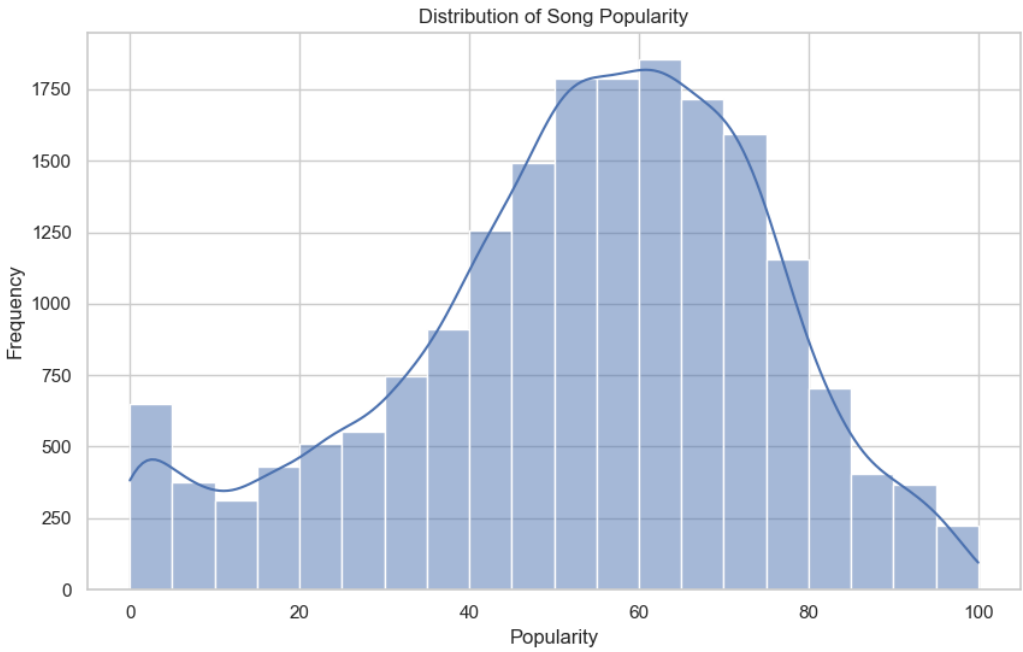
PRELIMINARY DATA ANALYSIS

After running **df.describe()** we find some basic statistics about the distribution of data.

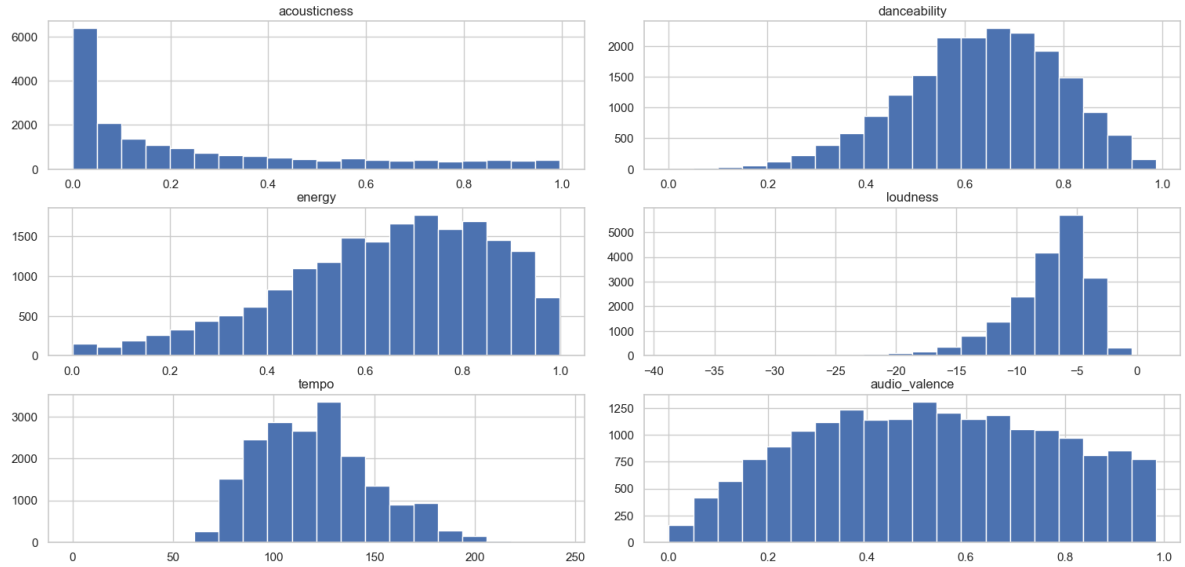
Mean song popularity is **52**, the danceability and energy features all have a mean of around **0.6**, which may mean the dataset is a bit biased towards more upbeat and energetic songs, the mean tempo is about **120** BPM, with a maximum of **242**.

Some features may necessitate a closer look. For example, it's becoming increasingly popular to use **double time** for songs, which means that a song may be at 240 BPM, but if it puts kicks and drums only on every second beat, it slows down the whole listening experience so it may actually be at 120 BPM. Of course, this is most used in Hip-Hop and Trap, but we should keep this in mind if we want to draw relevant conclusions.

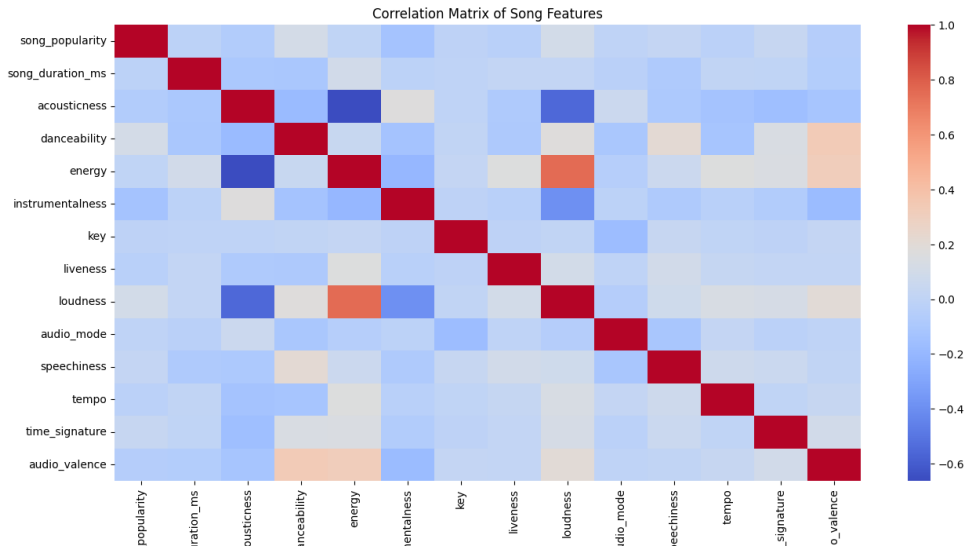
DISTRIBUTIONS



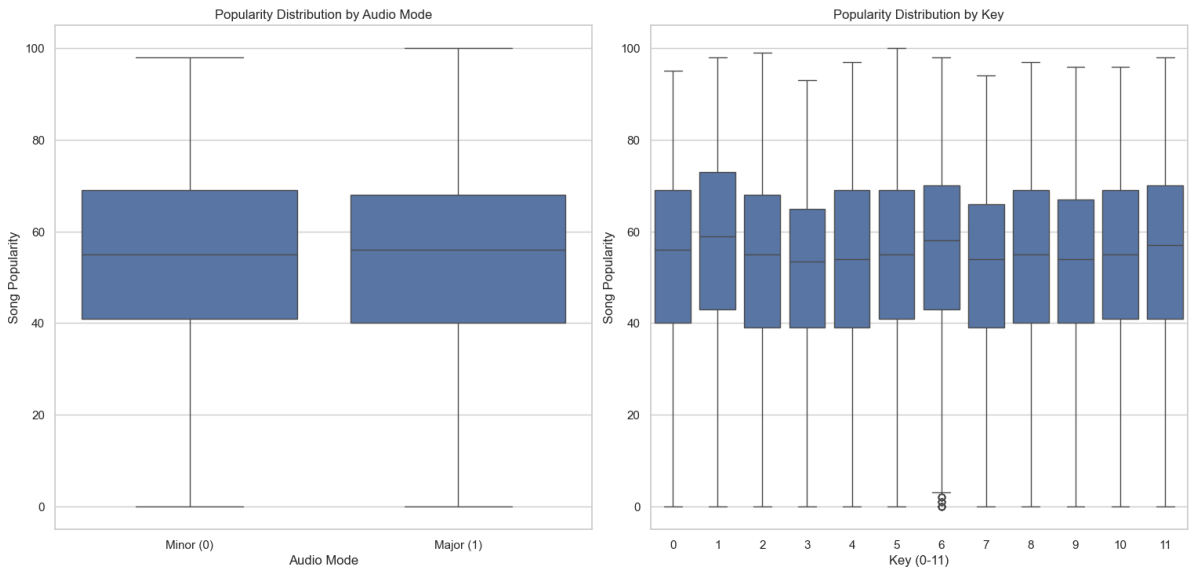
Song Popularity: The popularity score is not normally distributed. There's a significant number of songs with a popularity score of 0 and a peak around the 70-80 range. The high number of songs with zero popularity might be obscure tracks or data artifacts, which may be necessary to investigate or even filter out. It's also indicative of some contributors including their own music preferences, or forgetting to put enough unpopular songs in the dataset and they may have tried filling it up. Anyway, it's not great, but not a huge problem.



Other features: acousticness is heavily skewed to the right, loudness is left-skewed, but that totally makes sense. The other features appear to have distributions closer to normal.



The Correlation Matrix shows us expected results. Energy is closely linked to loudness and also the more acoustic a song is, usually the less energetic it is. The most significant correlation with **song_popularity** is a negative correlation: **the instrumentalness.** Here is our first real insight into the success of songs: popular songs are most likely not instrumental.



Categorical data, such as major or minor, or the song key doesn't give much on it's own to the popularity. Also, the song_name can be left out of the project, because the most used word that isn't a pronoun (*me, I, you*) or just a general word (*no, up, let, don't*) is **love**. It appears around **90** times in a dataset of 10.000+ records so the rest of the words don't count much towards calculating popularity. That is basically the only thing that the most common word analysis can tell us, so we are free to leave out the **song_name** from modeling.

INVESTIGATING THE 0 SONG POPULARITY

There are **272** songs with a popularity score of 0. I split the dataset into these 0 popularity songs and the rest. The distributions on the rest of the features basically line up. But, 36% of zero-popularity songs are highly instrumental, while in the other dataset only about 4% of the songs are highly instrumental.

They clearly skew the model's objective. My goal is to predict a song's popularity on a scale. A large cluster like this creates a conflict in the model. It will be forced to compromise between predicting the cluster at zero and predicting the distributed values, likely making it perform poorly on tests. Also, inside the zero-popularity songs are some of the biggest hits in the world, such as **„Waka Waka" by Shakira**, which suggests there was an error made in the dataset. I decided that I will need to remove these 272 records.

CONCLUSIONS AND NEXT STEPS

For the next milestone, I already know what to do:

- Remove duplicates by keeping first occurences
- Try to better understand double tempo in songs and account for it

- Remove songs with zero-popularity, because it's a clear error and the model would be heavily biased otherwise

I also drew some useful conclusions:

- I identified the target and feature columns
- What to include in the model and what isn't important or straight-up harmful if we include it (*song_name*)
- I will be focusing heavily on the audio features, such as instrumentalness, energy, etc.

All in all, it's not a perfect dataset, but a convenient one, with no missing values and only a small amount of work is needed to clean it up. For the next steps I'll be cleaning up the available information. I'll most likely also create bins to split songs into unpopular, mildly popular and very popular ones. This way we'll be able to analyze more closely how different features impact the success of a song.

Milestone 2: Data Preparation

INTRODUCTION AND REPRODUCIBILITY

The objective of this milestone was to clean the dataset, engineer features, and prepare the data completely so it's ready for modeling. A random seed (RANDOM_SEED = 66) throughout the notebook for reproducibility.

IMPROVING DATA QUALITY

Specific issues were addressed in Milestone 1, which we solved here:

- **Duplicate Removal**: We removed exact duplicate rows, keeping only the first occurrence.
- **Error Handling (Zero-Popularity)**: 272 tracks had a popularity score of 0 (including well-known songs like "Waka Waka"). These were determined to be collection errors and were dropped, as keeping them would confuse the model and heavily skew it.
- **Extreme Outlier Filtering**: I utilized the Interquartile Range (IQR) method specifically on the *song_duration_ms* feature. Tracks falling below Q1 - 1.5*IQR or above Q3 + 1.5*IQR were removed. **Why**: Extremely long (e.g., 15-minute) or short (e.g., 10-second) tracks are huge anomalies in popular music. Removing them ensures the model focuses on standard song structures. **Result**: The dataset was reduced to a clean set of ~14,000 samples.

FEATURE ENGINEERING

To capture true and useful characteristics, I designed and generated four new features (three hand-crafted and one adjusted) from the existing columns:

- **Intensity**: Calculated as *energy * loudness*. **Why**: This creates a single metric capturing the power of a track. Also, as we've seen the correlation matrix from Milestone 1, these features were highly correlated.
- **Party Vibe**: Calculated as *(danceability + energy) / 2*. **Why**: This represents the "club-readiness" or physical engagement level of a song.

- **Duration in Minutes**: Calculated as *song_duration_ms / 60000*. **Why**: Converts duration into a more interpretable scale.
- **Adjusted Tempo**: Tracks with a tempo strictly over 150 BPM were halved. **Why**: This resolves the "double-tempo" tracking algorithm flaw discovered in Milestone 1, normalizing the rhythmic distribution.

Combining features gives us a more useful set, while it also reduces dimensions and without losing essential information. Let's keep in mind the MECE principle (Mutually Exclusive and Collectively Exhaustive). This principle was developed by Barbara Minto at McKinsey & Company and is a principle mainly used in **management consulting**, but it's a useful way to think about feature engineering and selection: keeping the most unrelated features (grouping the correlated ones), that combined can give us a whole picture.

PREPARATION & DATA SPLITTING

Pre-processing (Conversions): To satisfy the sklearn API requirements, I dropped non-numeric string columns (*song_name*) and redundant columns that have a more useful alternative now (*tempo, song_duration_ms*). Categorical variables (key and time_signature) were converted into a machine-readable format using **One-Hot Encoding** (pd.get_dummies()).

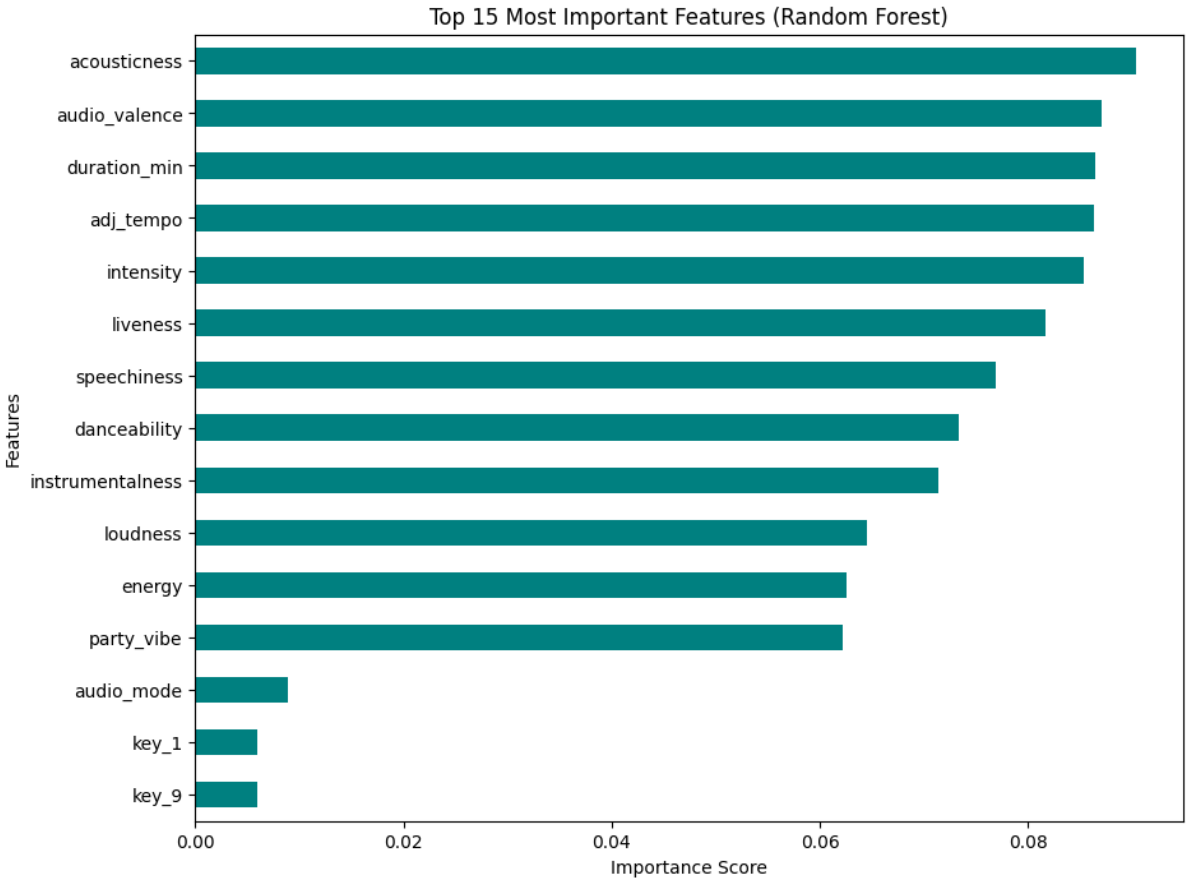
Train-Test Split: I split the dataset into Training (80%) and Testing (20%) sets.

- **Size**: Given our cleaned dataset of ~14,000 samples, an 80/20 split reserves over 11,000 tracks for learning complex patterns while maintaining ~2,800 tracks for evaluation.
- **Order of Operations**: The train/test split was performed strictly before any feature scaling, ensuring that metrics from the test set do not accidentally influence the training.

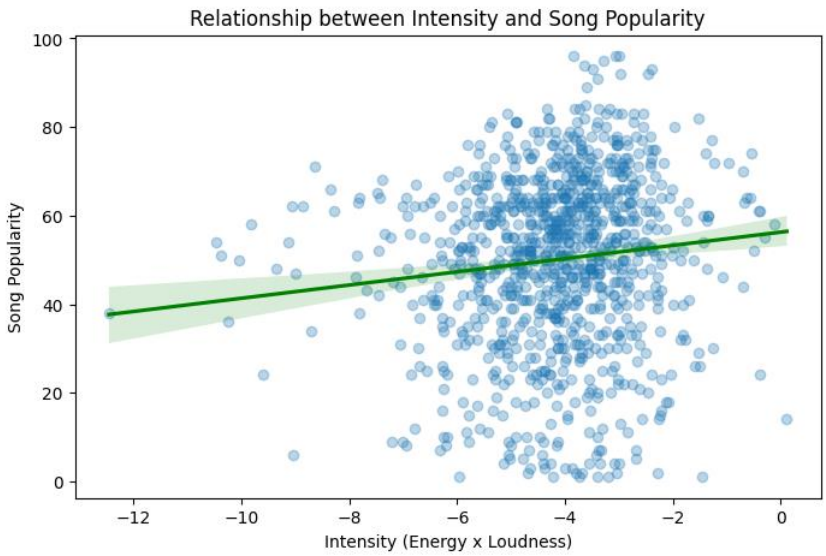
Feature Scaling: Continuous features (e.g., *loudness, duration_min, acousticness*) operate on vastly different numerical scales. We utilized **StandardScaler** to normalize them. **Why**: Scaling ensures that features with larger numerical values (*duration_min = 5 > acousticness = 0.4*) do not artificially dominate the metrics of the machine learning model. The scaler was fitted only on the training set, and then used to transform both the train and test sets.

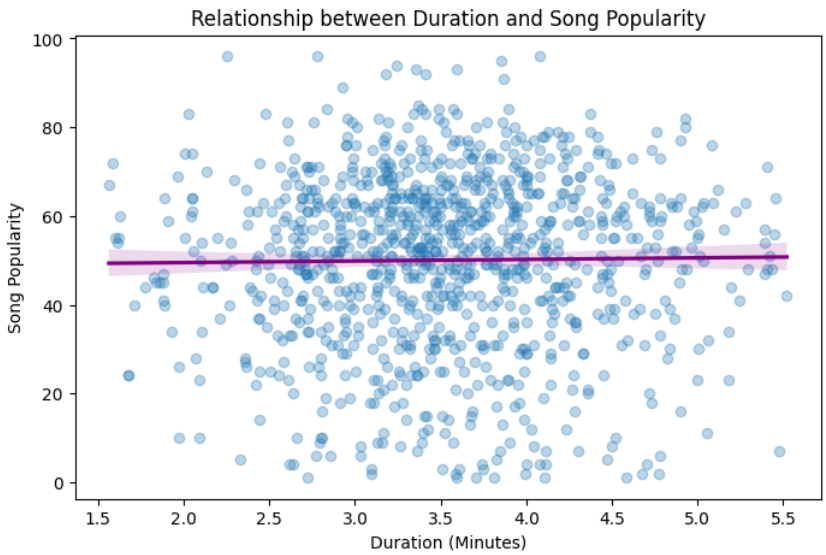
ANALYSIS OF NEW FEATURES

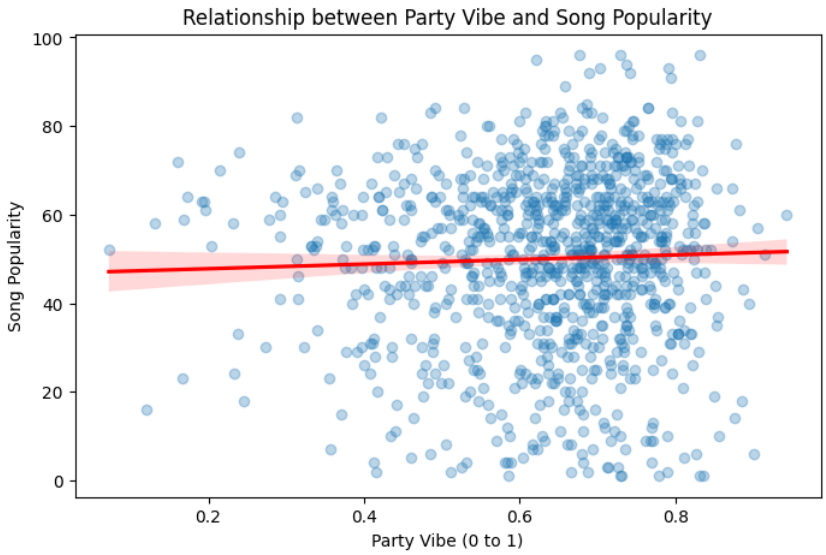
To determine which features hold the most predictive power, I used a **RandomForestRegressor** on the training data and extracted the feature importance scores.



The analysis revealed the strongest indicators of song popularity. Our *intensity* and *party_vibe* features ranked high, validating our feature engineering. Conversely, several of the One-Hot Encoded key variables showed near-zero importance and may be removed heading into Milestone 3. To better visualize these new features, I created a **Regression Plot** to see their relationship with our target variable:


Relationship between Intensity and Song Popularity
Song Popularity
Intensity (Energy x Loudness)
100
80
60
40
20
0
−12
−10
−8
−6
−4
−2
0


Relationship between Duration and Song Popularity
Song Popularity
Duration (Minutes)
100
80
60
40
20
0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
5.5


Relationship between Party Vibe and Song Popularity
Song Popularity
Party Vibe (0 to 1)
100
80
60
40
20
0
0.2
0.4
0.6
0.8

CONCLUSION

The objective of this milestone was to transform the data from Milestone 1 into a clean, machine-learning-ready dataset. I resolved important data quality issues. Our feature engineering efforts gave us new, predictive metrics. With the data cleaned, encoded, and scaled, we can continue to the modelling phase.

Milestone 3: Modelling

EVALUATION METRICS

Because the target variable (*song_popularity*) is a continuous numerical value ranging from 0 to 100, I treated this as a regression task. To accurately evaluate and compare our models, I selected two primary metrics:

- **Root Mean Squared Error (RMSE**): This metric measures the average distance between the model's predicted popularity and the actual true popularity. **Why**: RMSE heavily penalizes large errors and is in in the same units as our target (popularity points). For this metric, a **lower score is better** (less error).
- **R-squared (R^2)**: This metric indicates the proportion of the variance in the target variable that is explained by the model's features. **Why**: It provides a standardized scale of how well the model fits the data compared to a simple average guess. A baseline guessing strategy scores approximately 0.0, while a perfect predictor scores a 1.0. **Higher is better**.

BASELINE MODEL DEFINITION

To establish a minimum performance threshold, I implemented a **DummyRegressor** using the *mean* strategy.

- **What it does**: This baseline model completely ignores all audio features and simply predicts the mathematical average popularity of the training set for every single song in the test set.
- **Results**: The Dummy model yielded an **RMSE** of 20.01 and an R^2 of -0.000. This informs us that simply blindly guessing the average leaves us roughly 20 popularity points off the mark. Any valid machine learning model trained must achieve a lower **RMSE** and a positive R^2 to be considered successful.

MACHINE LEARNING MODELS SELECTED

To capture the unusual relationships between a song's properties and its popularity, I selected two robust, tree-based ensemble algorithms:

- **Random Forest Regressor**: An algorithm that builds hundreds of independent decision trees on varying subsets of the data and averages their outputs. This is resilient to overfitting.
- **Gradient Boosting Regressor**: A sequential algorithm where each new decision tree is specifically built to correct the residual errors made by the previous trees. It is highly effective.

Note on Reproducibility: Both models were trained on the scaled training set using a fixed random seed (66) to ensure complete reproducibility of all splits and algorithmic randomness.

HYPERPARAMETER TUNING

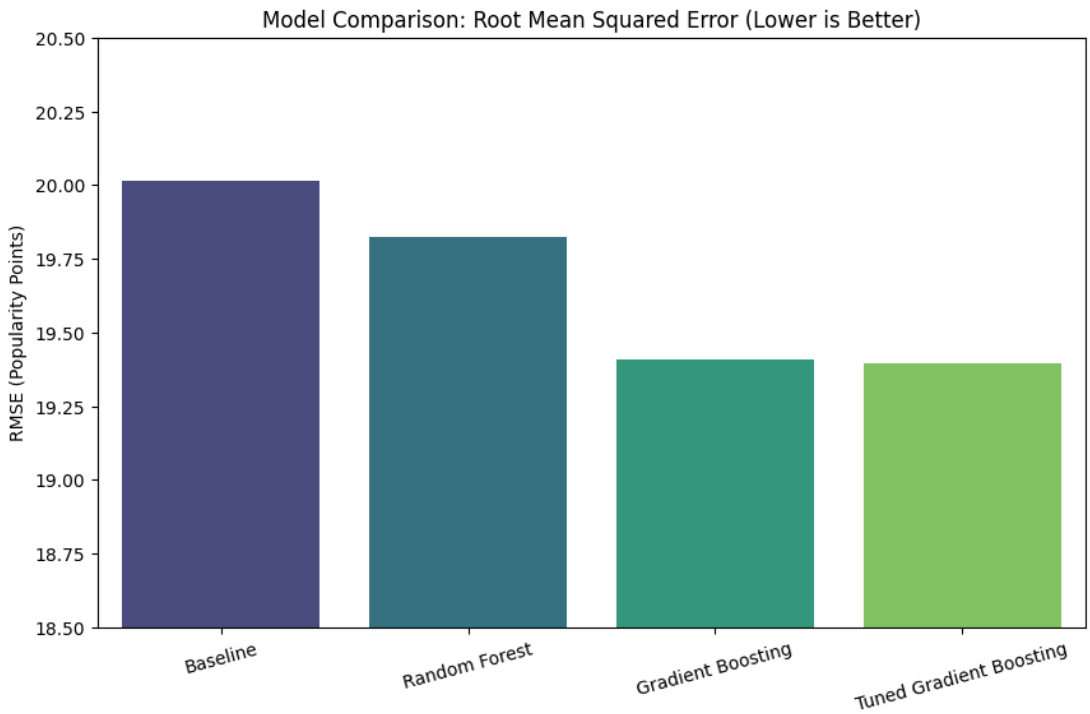
Initial evaluation showed that Gradient Boosting outperformed Random Forest (**RMSE** of 19.40 vs 19.86). I isolated the Gradient Boosting model and applied **GridSearchCV** to perform automated hyperparameter tuning.

- **What we did**: I utilized 3-fold cross-validation to search for the optimal combination of n_estimators (number of trees: 100 vs. 200), learning_rate (step size: 0.05 vs. 0.1), and max_depth (tree complexity: 3, 4, 5).
- **Results**: The Grid Search optimized for the lowest error and identified the best parameters as n_estimators=100, learning_rate=0.1, and max_depth=4.

FINAL EVALUATION AND COMPARISON

Final Model Performance on Test Set

Model	RMSE	R^2
Baseline (Mean Guess)	20.01	-0.000
Random Forest	19.86	0.014
Gradient Boosting	19.40	0.059
Tuned Gradient Boosting	19.39	0.061



INTERPRETATION OF RESULTS

The machine learning models successfully beat the baseline strategy. The Tuned Gradient Boosting model performed the best overall, reducing the error to an **RMSE** of 19.39 and explaining roughly 6% of the variance ($\boldsymbol{R^2}$ of 0.061).

While an $\boldsymbol{R^2}$ of 6% is relatively low, it is expected outcome given the constraints of the dataset. We are attempting to predict human subjectivity utilizing only mathematical properties (e.g., loudness, tempo). Critical variables such as artist fame, marketing budgets, social media trends, and release dates are completely absent from the dataset, as I noted in Milestone 1. However, achieving a positive $\boldsymbol{R^2}$ and consistently lowering the **RMSE** confirms that the model is successfully detecting existing patterns that slightly influence a track's success.